

MixBytes()

[SECURITY AUDIT REPORT]

Lido L2 Direct Staking (Linea Adapter)

PREPARED FOR LIDO

REPORT DATE

August 04, 2026

Table of Contents

1. Introduction	3
1.1 Disclaimer	3
1.2 Executive Summary	3
1.3 Project Overview	5
1.4 Security Audit Methodology	7
1.5 Risk Classification	9
1.6 Summary of Findings	10
2. Findings Report	10
2.1 Critical	10
2.2 High	10
2.3 Medium	10
2.4 Low	11
L-1 PriceOracle reads the Chainlink feed on L2 without consulting the L2 Sequencer Uptime Feed, so a stale price can be served during the sequencer-recovery window	11
3. About MixBytes	12

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

The reviewed scope is part of a Chainlink CSR-based L2 direct-staking system that lets users on supported L2s stake native assets and receive a liquid-staking token, using Chainlink CCIP as the cross-chain transport and per-chain canonical bridges for delivering the staked token back to L2. The `CustomReceiver` contract receives the inbound CCIP message on L1, unwraps and stakes the native asset, and then delegate-calls a chain-specific `BridgeAdapter` to forward the staked token to the recipient on the destination L2. `LineaAdapterL1toL2` is the Linea-specific adapter: it approves and calls `bridgeToken` on the Linea Canonical Token Bridge to move the staked token from L1 to Linea. Adapters are stateless by design and are invoked exclusively via `delegatecall` from the delegator to avoid storage collisions.

The interim security review was conducted over 1 working day via manual code review and a proprietary AI-assisted analysis tool.

Our focus spanned both the core adapter logic and its interaction with the external Linea Canonical Token Bridge and the delegator that drives it. Alongside the protocol-specific analysis, we worked through our standard checklist covering access control and the `delegatecall` /storage-collision boundary, fee-data encoding/decoding, approval handling, reentrancy, integer handling, and cross-chain delivery assumptions. In particular, the following project-specific attack vectors were investigated during this engagement:

- **Asynchronous L2 credit vs. recorded amount.** The Linea Canonical Token Bridge measures the actually received token amount (escrow balance delta) and mints only that net amount on L2, while the adapter forwards the caller-supplied `amount` unchecked. We investigated whether any fee-on-transfer, rebasing, or non-standard behavior of the staked token could cause the amount credited on Linea to diverge from the amount recorded/emitted on L1, and whether the system anywhere assumes atomic or exact L2 crediting. In the current configuration the bridged token is a standard, non-fee-on-transfer ERC-20, so this reduces to a token assumption to be documented rather than an exploitable divergence.
- **Bridge-operator liveness coupling on `bridgeToken` failure.** `_sendToken` is executed synchronously inside `_processMessage` via `delegatecall`, so a revert from `bridgeToken` (token marked `RESERVED` /not deployed, `INITIATE_TOKEN_BRIDGING` paused by a bridge role, or the message service paused) fails the inbound token-forwarding leg. Tracing the full path, we confirmed this failure is caught by the L1 `CCIPDefensiveReceiver`, which parks the tokens for owner recovery (`recoverTokens`) rather than losing them - an explicit escape hatch exists. We assessed the residual as a liveness/operational concern (delivery stalls until Linea unpauses or the owner recovers), not a fund-loss or permanent-strand condition.
- **Recipient integrity of the L2->L1->L2 round-trip.** We traced where the `recipient` / `to` handed to `bridgeToken` originates. It is not attacker-controlled: the CRE report that drives the flow is nullary (`CREReceiver` enforces argument-less, allow-listed calls), and `CustomSender` hard-sets the CCIP `recipient` to the configured OraclePool, so no crafted recipient can be injected. We instead examined owner-misconfiguration of the OraclePool/recipient and the fact that the Linea bridge reverts on `address(0)` or reserved/undeployed token state, confirming such cases fail into the recoverable path rather than griefing other users.
- **Split fee-blob validation across L2 and L1.** The Linea `feeDto0` blob the adapter decodes with `FeeCodec.decodeLineaL1toL2` is authored on L2 by `SyncTrigger`, whose `_setFeeDto0` is deliberately generic (enforces only length ≥ 17 and rejects `payInLink`) and does NOT pin the Linea 17-byte / zero-feeAmount shape - the adapter is the sole on-chain enforcement point. We reviewed this split for gaps: a lane-mismatched or non-zero-fee Linea blob is accepted on L2, causes `CustomSender` to pull the encoded fee amount and cross CCIP, then reverts inside the adapter (`InvalidFeeAmount` / `InvalidFeeToken` /length check) into the recoverable receiver path. We also reviewed the `forceApprove(TOKEN_BRIDGE, amount)` / `bridgeToken` sequence for residual-allowance or partial-consumption risk, and the reliance on Linea's off-chain postman sponsorship for auto-claim on L2. The residual is an owner-misconfiguration cost (one wasted, L1-recoverable sync round-trip) rather than a loss of funds.

The Linea Canonical Token Bridge (`ILineaTokenBridge` / `TOKEN_BRIDGE`), the underlying Linea Message Service, the `CustomReceiver` staking/CCIP path, and Chainlink CCIP infrastructure are external to the adapter and were not audited line-by-line as part of this engagement; our assessment of them was limited to the integration points exercised by in-scope code and the bridges' published interfaces and documentation.

At the Client's request, the scope was subsequently extended (at commit `34344f5b247de110c74c18df6b445ab1cb238f5e`) to cover the L2 pricing subsystem: `PriceOracle` (a Chainlink `AggregatorV3Interface` wrapper that normalizes to 1e18 scale and enforces staleness/positivity), `PriceConverterOracle` (which composes two price oracles), their interfaces (`IPriceOracle`, `IPriceConverterOracle`, `IOracle`), and the `OraclePool` contract that consumes the oracle price to swap the user's native asset for the staked token via `CustomSender.fastStake`. These contracts are not on the Linea adapter's L1 inbound path; they were pulled in together with their consumer because the exploitable surface of the oracle price lives in how `OraclePool` applies it, not in the oracle wrappers themselves. The external Chainlink data feed (`AggregatorV3Interface`) remains a documented trust assumption and was not audited line-by-line.

Below we set out our overall assessment, key assumptions, and main recommendations.

- **Focused, minimal adapter surface.** `LineaAdapterL1toL2` is small and single-purpose: it decodes fee data, enforces the zero-fee/native-fee invariant, approves, and calls `bridgeToken`. The stateless-adapter pattern with the `onlyDelegatedByDelegator` guard and immutables (no storage) correctly avoids storage collisions with the delegator, which is the main structural risk of the `delegatecall` design.
- **External-delivery assumptions concentrate the risk.** The security of the flow depends on Linea-bridge behavior that is not enforced on-chain here - zero postman fee, off-chain auto-claim sponsorship, and net-amount accounting. These are reasonable given current Linea behavior but should be documented as explicit trust assumptions, since a change (fee-on-transfer token, sponsorship policy change, or bridge pause) shifts outcomes from "delivered automatically" to "must be manually claimed" or reverts the staking flow.
- **No escape hatch on synchronous bridge failure.** A revert inside `bridgeToken` unwinds the whole inbound staking transaction; consider whether a design that can retry/queue the L2 send (rather than reverting the stake) is warranted, and confirm the intended operational response if Linea pauses initiate-bridging.
- **Recommendations.** Document the assumption that the staked token is a standard, non-fee-on-transfer ERC-20 and is not reservable on Linea; validate/constrain the `recipient` upstream where possible; and add explicit test coverage for paused-bridge and reserved-token behavior of the Linea path.
- **Oracle staleness vs. zero pool fee.** The oracle wrappers are sound for the deployed feeds, but `OraclePool.swap` prices against a value up to `HEARTBEAT` (24h) stale while the pool `fee` - the only buffer against that lag - is 0 on all chains. Since the `wstETH/stETH` rate only rebases upward, a user can swap at the pre-rebase rate inside the heartbeat window. We recommend setting `ORACLE_POOL_FEE` to at least the per-heartbeat rebase drift and/or shortening `HEARTBEAT`. This is a configuration risk at the oracle-to-pool boundary, not a flaw in the oracle contracts.
- **Single-EOA ownership over live pool inventory.** Every `OraclePool` is owned by the same deployer EOA on all four chains (`*_OWNER` defaults to `address(0)`), and `sweep` is `onlyOwner` with no token restriction - unlike `pull`, which is limited to `TOKEN_IN` - so that one key can drain all `TOKEN_OUT` (`wstETH`) inventory and pause the pools. The exposure is operator liquidity, not user deposits (`fastStake` is atomic and `minAmountOut`-bounded), so worst case is operator loss plus temporary DoS. We recommend moving pool ownership and the `CustomSender` `admin` / `SYNC_ROLE` to a multisig or timelock, setting `*_OWNER` explicitly, and optionally barring `sweep` of `TOKEN_OUT`.
- **Cached feed decimals.** `PriceOracle` caches `DECIMALS` from the Chainlink proxy at construction and scales every read by `10 ** DECIMALS`. If a future aggregator migration behind the proxy reported different decimals, prices would mis-scale silently while all checks (positivity, heartbeat) still pass. Chainlink keeps decimals constant across migrations (currently 18). We recommend reading `decimals()` at call time or documenting the fixed-decimals assumption and monitoring Chainlink phase migrations.
- **Cross-feed timestamp inconsistency in `PriceConverterOracle`.** `getLatestAnswer` multiplies two independent `PriceOracle` reads without reconciling their `updatedAt` timestamps; each leg only enforces its own heartbeat, so one feed can be fresh while the other is up to `HEARTBEAT` (24h) old. The composed price then combines values from different points in time and can drift transiently (e.g. an `A:B x B:C` pair whose product briefly deviates from the true rate) until both legs catch up. This does not affect the current Lido deployment - every pool uses a single `PriceOracle` and the converter is not deployed - but if the converter is ever used it should either bound the timestamp spread between the two legs or document the residual as an accepted assumption.
- **NatSpec inaccuracies.** Several comments contradict their code and should be corrected to prevent misconfiguration: `OraclePool.swap` describes the oracle price in the inverse direction to what the code divides by; `PriceConverterOracle` states the quote-leg orientation opposite to its own `A:B x B:C` example (a deployer following the prose could wire the legs backwards); `PausableImmutableOraclePool.swap` carries a leftover copy of `sweep`'s NatSpec; and `CustomSender.setOraclePool` claims it approves/revokes WNative allowances, but `_setOraclePool` only writes storage (the approval is done per-call in `fastStake`).

1.3 Project Overview

Summary

CLIENT	Lido
CATEGORY	Liquid Staking
PROJECT	L2 Direct Staking (Linea Adapter)
TYPE	Solidity
PLATFORM	EVM
TIMELINE	06.07.2026 - 21.07.2026

Scope of Audit

contracts/adapters/LineaAdapterL1toL2.sol	contracts/libraries/FeeCodec.sol	contracts/utils/PriceOracle.sol
contracts/utils/PriceConverterOracle.sol	contracts/interfaces/IPriceOracle.sol	
contracts/interfaces/IPriceConverterOracle.sol	contracts/interfaces/IOracle.sol	

Versions Log

DATE	COMMIT HASH	NOTE
06.07.2026	62108f7b6cc664e36dbc8100c4b48974d59f572e	Initial Commit
21.07.2026	34344f5b247de110c74c18df6b445ab1cb238f5e	Commit with Updates

Mainnet Deployments

FILE	ADDRESS	BLOCKCHAIN
LineaAdapterL1toL2.sol	0x122beD1eB48DC4679DDF2C8fc159e9c498344397	Ethereum
PriceOracle.sol	0x328de900860816d29D1367F6903a24D8ed40C997	Arbitrum
PriceOracle.sol	0x301cBCDA894c932E9EDa3Cf8878f78304e69E367	Optimism
PriceOracle.sol	0x301cBCDA894c932E9EDa3Cf8878f78304e69E367	Base
PriceOracle.sol	0x301cBCDA894c932E9EDa3Cf8878f78304e69E367	Linea

LineaAdapterL1toL2 deployment verified byte-for-byte against the audit-scope source at commit 62108f7b6cc664e36dbc8100c4b48974d59f572e, block 22932646.

Constructor arguments:

PARAMETER	VALUE	NOTES
tokenBridge	0x051F1D88f0aF5763fB888eC4378b4D8B29ea3319	Linea: L1 Token Bridge (canonical)
token	0x7f39C581F595B53c5cb19bD0b3f8dA6c935E2Ca0	wstETH (mainnet)
delegator	0x6F357d53d6bE3238180316BA5F8f11467e164588	CustomReceiver (delegator proxy)

All three read back from the on-chain getters (`TOKEN_BRIDGE`, `TOKEN`, `DELEGATOR`) as the values above. The adapter is correctly bound into the L1 receiver: `LidoCustomReceiver.getAdapter(4627098889531055414)` (the Linea CCIP chain selector) returns this adapter, and the receiver proxy's ERC-1967 implementation and admin slots hold `0x301cBCDA894c932E9EDa3Cf8878f78304e69E367` and `0x88a45d2760b63c1500E3D2E3552b28e5Cdaa37BD` respectively, matching the address list published in [lidofinance/docs#957](#). The adapter is immutable and stateless.

The four `PriceOracle` deployments are likewise verified byte-for-byte against the audit-scope source at commit 34344f5b247de110c74c18df6b445ab1cb238f5e, with two distinct but equally-verified build profiles, which is the one deviation worth recording:

LANE	DEPLOYMENT BLOCK	RUNTIME SIZE	OPTIMIZER	AGGREGATOR (CHAINLINK WSTETH/STETH FEED)
Arbitrum	259088005	1371 bytes	enabled, runs = 200	0xB1552C5e96B312d0Bf8b554186F846C40614a540
Optimism	126071346	1371 bytes	enabled, runs = 200	0xe59EBa0D492cA53C6f46015EEa00517F2707dc77
Base	20476080	1371 bytes	enabled, runs = 200	0xB88BAc61a4Ca37C43a3725912B1f472c9A5bc061
Linea	20930299	2310 bytes	disabled	0x3C8A95F2264bB3b52156c766b738357008d87cB7

Arbitrum/Optimism/Base were compiled with the optimizer on and Linea with it off, so the Linea instance is larger despite being the same contract. Both profiles were reproduced from the audited source and match their respective deployments.

The remaining immutables are uniform and correct on all four lanes: `IS_INVERSE = false`, `DECIMALS = 18`, `HEARTBEAT = 86400` (24h). Each `AGGREGATOR` is the correct per-chain Chainlink wstETH/stETH feed for its lane and matches the feed list published in the Lido deployed-contracts page. At the time of verification all four `getLatestAnswer()` calls returned the same live value (`1.240921811464463804e18`), consistent with a healthy, non-stale feed on every lane. `PriceOracle` is immutable and ownerless, so these constructor values are its complete and unchangeable configuration; the only residual operational dependency is feed liveness, which the 24h `HEARTBEAT` converts into a revert (`PriceOracleStalePrice`) rather than a stale-price read.

1.4 Security Audit Methodology

Stage 1 Interim Audit

01 PROJECT ARCHITECTURE REVIEW

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING THE PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

- Understanding overall system design and contract interactions
- Mapping responsibilities of each component and protocol workflows
- Conducting a thorough line-by-line review of contracts and modules
- Tracing execution paths and mapping state transitions
- Identifying trust boundaries and external integrations
- Forming an independent architectural understanding directly from code before validating documentation
- Running the project test suite to observe behavior and encoded invariants

02 ADVERSARIAL CODE REVIEW

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

- Analyzing the codebase from an attacker's perspective: what can fail, not only what works
- Searching for edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- Attempting to violate protocol invariants and find where guarantees break
- Analyzing economic attack surfaces: oracle dependencies, share pricing, access control, cross-contract interactions
- Writing targeted tests, modelling adversarial scenarios, mutation testing, fuzzing, and PoC exploits
- Independent exploit-path discovery by each auditor to reduce anchoring bias
- Collaborative pair-auditing of high-risk code led by the Team Lead

03 SYSTEMATIC VULNERABILITY ANALYSIS

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

- Reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- Analyzing common DeFi attack classes: reentrancy, accounting manipulation, oracle manipulation, privilege escalation, state desynchronization
- Using proprietary AI tooling to surface candidate issues across broader attack vectors
- Manually validating and triaging AI-generated candidates

04 CONSOLIDATION OF AUDITORS' REPORTS

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

- Cross-checking findings across auditors; consolidating and deduplicating issues
- Resolving discrepancies in severity assessments
- Using proprietary AI tooling to assist with report drafting and consistency checks
- Manually reviewing and finalizing the report narrative for client review

Stage 2

Re-audit & Mainnet Verification

RE-AUDIT

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THE UPDATED CODE INTRODUCES NO NEW VULNERABILITIES.

- Confirming each remediation matches the original recommendation
- Documenting rationale for any unresolved findings
- Verifying modified logic and integration points remain secure
- Executing the test suite after remediation, targeting fixed areas
- Running AI tooling on the updated codebase

MAINNET DEPLOYMENT VERIFICATION

OBJECTIVE: FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC REPORT.

- Verifying deployed bytecode against audited source across networks
- Matching bytecode to the build from the audited commit, identical compiler settings
- Reviewing constructor and initializer arguments
- Verifying proxy order, initialization flow, and admin configuration
- Ensuring implementations aren't left uninitialized or exposed to init front-running

1.5 Risk Classification

Severity Level Matrix

SEVERITY	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
LIKELIHOOD: HIGH	CRITICAL	HIGH	MEDIUM
LIKELIHOOD: MEDIUM	HIGH	MEDIUM	LOW
LIKELIHOOD: LOW	MEDIUM	LOW	LOW

IMPACT

HIGH - Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.

MEDIUM - Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.

LOW - One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

LIKELIHOOD

HIGH - An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g. due to a market condition that the actor cannot influence).

MEDIUM - An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.

LOW - A highly unlikely event that can only be triggered by the contract owner.

ACTION REQUIRED

CRITICAL - Must be fixed as soon as possible.

HIGH - Strongly advised to be fixed in order to minimize potential risks.

MEDIUM - Recommended to be fixed to improve security and stability.

LOW - Recommended to be fixed to improve overall robustness and efficiency.

FINDING STATUS

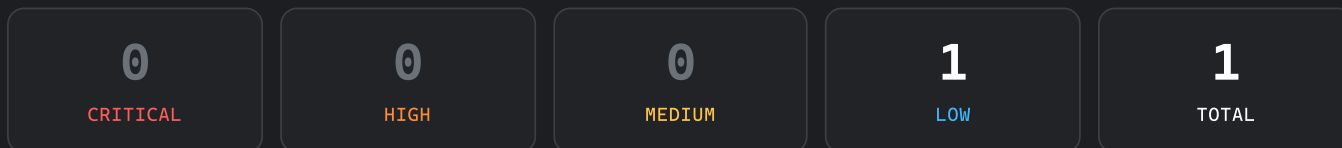
● **FIXED** - The recommended fixes have been implemented and the issue no longer impacts the security of the protocol.

● **PARTIALLY FIXED** - The fixes have been partially implemented, reducing the impact, but the issue has not been fully resolved.

● **ACKNOWLEDGED** - The fixes were not implemented; the finding is unresolved or was accepted by the team without code changes.

1.6 Summary of Findings

Findings Count



Findings Statuses

ID	FINDING	SEVERITY	STATUS
L-1	PriceOracle reads the Chainlink feed on L2 without consulting the L2 Sequencer Uptime Feed, so a stale price can be served during the sequencer-recovery window	● LOW	● ACKNOWLEDGED

2.1 Critical

NO CRITICAL SEVERITY FINDINGS

2.2 High

NO HIGH SEVERITY FINDINGS

2.3 Medium

NO MEDIUM SEVERITY FINDINGS

2.4 Low

L-1 PriceOracle reads the Chainlink feed on L2 without consulting the L2 Sequencer Uptime Feed, so a stale price can be served during the sequencer-recovery window

● LOW

● ACKNOWLEDGED

DESCRIPTION

`PriceOracle.getLatestAnswer ( contracts/utis/PriceOracle.sol )` runs exclusively on L2 rollups but prices the feed with only a staleness/positivity check - no L2 Sequencer Uptime Feed check:

```
if (answer <= 0) revert PriceOracleInvalidPrice();
if (block.timestamp > updatedAt + HEARTBEAT) revert PriceOracleStalePrice();
```

When an L2 sequencer goes down and restarts, users can transact again before the feed posts a fresh round. If the total staleness is still within `HEARTBEAT` (24h), the pre-outage price passes the check and is served as fresh. Chainlink's L2 Sequencer Uptime Feed exists precisely to reject prices during/just after downtime; `PriceOracle` never reads it.

This issue is identified as Low: all four live feeds are slow `wstETH/stETH` exchange rates, so the mispricing across even a full 24h outage is approx $\leq 0.011\%$, bounded by pool inventory and operator-borne (not user funds), and an outage exceeding the 24h heartbeat fails closed (`PriceOracleStalePrice`).

RECOMMENDATION

Read the L2 Sequencer Uptime Feed in `getLatestAnswer` and revert while the sequencer is down or within a grace period after restart.

Built on Trust

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems. The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle - from early architecture design to production audits and ongoing protocol evolution. Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including Lido, Aave, Curve, 1inch, OKX, Mantle, Fluid, Gearbox, Resolv, and others. To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review.

SECURED TVL

\$50B+

AUDITS DELIVERED

300+

WEB3 PROTOCOLS

80+

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

Web-3 Design Review

Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.

AI Tooling

AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.

Smart Contract Audit

Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.

Security Support

Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

CONTACTS

hello@mixbytes.io

[X](#) [Telegram](#) [Discord](#) [LinkedIn](#)